



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

PROJECT 2 (UNSUPERVISED LEARNING) AND REFLECTION

Course : SECP2753 DATA MINING

Semester/Session : 2-2024/2025

Section : 02

Lecturer : Dr Rozilawati binti Dollah @ Md Zain

No.	Group Members	Matric No.
1.	Muhammad Adam bin Razali	A23CS0116
2.	Iman Abadi bin Mohd Nizwan	A23CS0084
3.	Muhammad Afiq Danish bin Mohd Hazni	A23CS0118

1.0 INTRODUCTION

In today's data-driven era, analyzing student performance has become essential for identifying academic trends and enhancing institutional support systems. By leveraging unsupervised learning techniques, we can uncover underlying patterns in student behavior and academic performance that may not be visible through traditional analysis. This project utilizes clustering methods to explore a dataset containing various factors influencing student performance, such as attendance, study hours, parental involvement, and access to resources.

Unsupervised learning, particularly clustering, is well-suited for this task as it does not rely on predefined labels. Instead, it allows for the natural grouping of data based on similarities among features. This enables the discovery of student segments that may share common characteristics or challenges. For this project, we employed two common clustering algorithms which are KMeans and Agglomerative Clustering. Both methods offer unique approaches to grouping data and can provide valuable insights when compared.

The primary aim of this project is to uncover meaningful groupings within the student population that can inform better academic strategies. By identifying these clusters, educators and administrators can make more informed decisions in tailoring interventions, allocating resources, and improving student outcomes. The following sections describe our methodology, including preprocessing steps, model construction, results, and discussions.

2.0 DATA PREPROCESSING METHODS

In this project, we conducted a series of preprocessing steps on the “Student Performance” dataset, which contains a mix of numerical and categorical features describing various academic and behavioral factors of students. The primary goal of these steps was to handle missing values, remove outliers, encode categorical variables, normalize numerical data, and perform feature selection to ensure optimal clustering results.

2.1 Categorizing the Features

The dataset included both numerical and categorical variables, some of which contained missing values. First, we categorize the features into two which are categorical and continuous. The categorical features are stored in the *categorical_features* variable while continuous (numerical) features are stored in the *continuous_features* variable.

```
# Categorizing the features
categorical_features = ['Extracurricular_Activities', 'Internet_Access', 'School_Type', 'Learning_Disabilities', 'Gender',
                        'Parental_Involvement', 'Access_to_Resources', 'Motivation_Level', 'Family_Income', 'Teacher_Quality', 'Peer_Influence',
                        'Parental_Education_Level', 'Distance_from_Home']

continuous_features = ['Hours_Studied', 'Attendance', 'Previous_Scores', 'Sleep_Hours', 'Tutoring_Sessions', 'Physical_Activity', 'Exam_Score']
```

Figure 2.1: Categorizing the Features

Figure 3.1 shows the features that are being stored in *categorical_features* variable such as **Extracurricular_Activities**, **Internet_Access**, **School_Type** and others. Not to mention, *continuous_features* variable stored *numerical features* such as **Hours_Studied**, **Attendance**, **Previous_Scores** and many more.

2.2 Handle Missing Values

We handled missing values using the *SimpleImputer* from the *sklearn* library. Numerical features were imputed using the mean strategy (*strategy* = ‘mean’), replacing missing values with the average of the respective column. Categorical features were imputed using the most frequent strategy (*strategy* = ‘most_frequent’), filling in missing entries with the most commonly

occurring category. This step ensured that the dataset was complete and suitable for further processing.

```
num_imputer = SimpleImputer(strategy='mean')
cat_imputer = SimpleImputer(strategy='most_frequent')

dataset_copy[continuous_features] = num_imputer.fit_transform(dataset_copy[continuous_features])
dataset_copy[categorical_features] = cat_imputer.fit_transform(dataset_copy[categorical_features])
```

Figure 2.2: Impute Missing Values for both Numerical and Categorical Features

2.3 Outlier Removal

To enhance data reliability and reduce the influence of extreme values, we applied outlier removal using the **Interquartile Range (IQR)** method. For each continuous feature, we calculated the first (Q1) and third quartiles (Q3) and determined the interquartile range ($IQR = Q3 - Q1$). Any data point falling below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$ was considered an outlier and removed from the dataset. This filtering process was repeated for all continuous features to ensure that only valid and representative data points were retained.

```
for col in continuous_features:

    q1 = dataset_copy[col].quantile(0.25)
    q3 = dataset_copy[col].quantile(0.75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    dataset_copy = dataset_copy[(dataset_copy[col] >= lower_bound) & (dataset_copy[col] <= upper_bound)]
```

Figure 2.3: Handling Outliers in Continuous Features

2.4 Encoding Categorical Features

For the next step, we proceeded to encode categorical variables so that they could be interpreted by clustering algorithms. Most machine learning algorithms require numerical input, and thus categorical features must be transformed appropriately. We used One-Hot Encoding (OHE), a common encoding technique that converts each category into a new binary column. This encoding was applied to variables such as **"Internet_Access"**, **"Extracurricular_Activities"**, **"Parental_Education_Level"**, and several others. One-hot encoding prevents the algorithm from assuming any ordinal relationship among categories, which could otherwise introduce biases. The process was performed using the *OneHotEncoder* function from the *sklearn* library, with *sparse_output=False* and *handle_unknown='ignore'* to ensure compatibility and robustness.

```
ohe = OneHotEncoder(sparse_output = False, handle_unknown = 'ignore')
encoded_categorical = ohe.fit_transform(dataset_copy[categorical_features])
encoded_categorical = pd.DataFrame(encoded_categorical, columns = ohe.get_feature_names_out(categorical_features))
```

Figure 3.4: Encoding Categorical Features

2.5 Feature Scaling

Next, we scaled the numerical features using *StandardScaler* function from *sklearn* library, which standardized the data by centering it around a mean of zero and a standard deviation of one. This step was essential because clustering algorithms are sensitive to the magnitude of input features. Features with larger scales could otherwise dominate the clustering process.

```
scaler = StandardScaler()
scaled_continuous = scaler.fit_transform(dataset_copy[continuous_features])
scaled_continuous = pd.DataFrame(scaled_continuous, columns = continuous_features)
```

Figure 3.5: Scaling Numerical Features

2.6 Feature Selection

For the last step, we performed feature selection using the **Variance Threshold** method. Not all features contribute equally to the clustering task, some may have very low variance and provide little to no difference between data points. To identify and remove such features, we applied the *VarianceThreshold* function from *sklearn* library, setting the threshold to 0.9. This means that any feature with a variance lower than 0.9 was excluded from the dataset. By reducing dimensionality, this technique not only speeds up computation but also helps improve clustering performance by focusing on the most informative attributes. A list of selected features was printed and used as input to the clustering models.

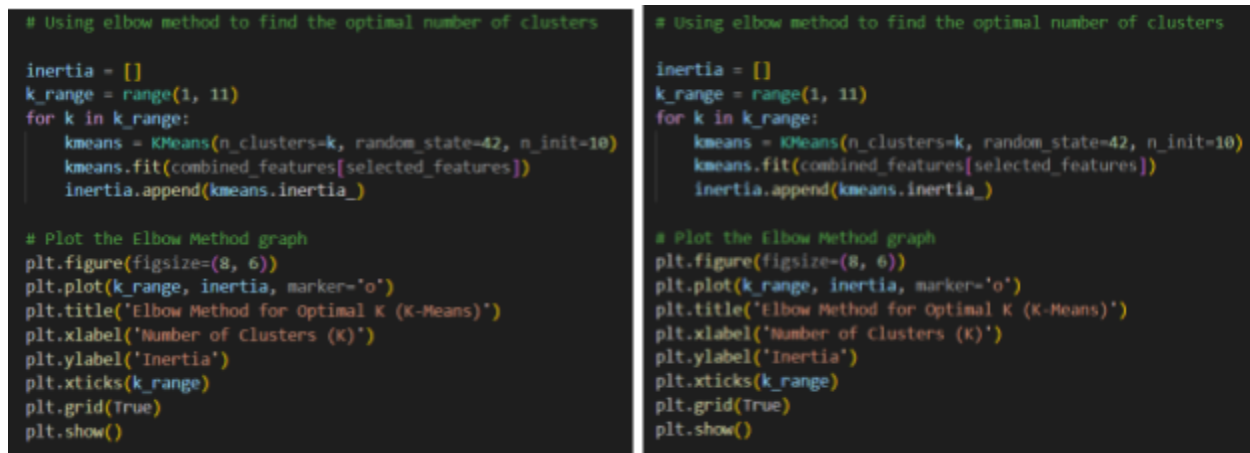
```
selector = VarianceThreshold(threshold = 0.9)
selected_features_indices = selector.fit_transform(combined_features)
selected_features = combined_features.columns[selector.get_support()].to_list()

print(selected_features) # 6 important features selected
```

Figure 2.6: Feature Selection

3.0 CONSTRUCTION OF MODELS

For the construction of the clustering models, we decided to choose partitioning algorithms using *K-Means clustering* and Hierarchical algorithms using *Agglomerative clustering*. For each of the clustering algorithm we employs the Elbow method to determine the optimal number of clusters K . It iterates through a range of K values (from 1 to 10), fits a K -Means model for each K , and calculates the inertia (within-cluster sum of squares). These inertia values are then plotted against the number of clusters, creating an "elbow" graph. The "elbow point" on this graph typically suggests the optimal K , where adding more clusters no longer significantly reduces inertia in. Therefore we decided on taking $K = 3$ for the number of clusters of each clustering algorithm.



```
# Using elbow method to find the optimal number of clusters
inertia = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(combined_features[selected_features])
    inertia.append(kmeans.inertia_)

# Plot the Elbow Method graph
plt.figure(figsize=(8, 6))
plt.plot(k_range, inertia, marker='o')
plt.title('Elbow Method for Optimal K (K-Means)')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.xticks(k_range)
plt.grid(True)
plt.show()
```

```
# Using elbow method to find the optimal number of clusters
inertia = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(combined_features[selected_features])
    inertia.append(kmeans.inertia_)

# Plot the Elbow Method graph
plt.figure(figsize=(8, 6))
plt.plot(k_range, inertia, marker='o')
plt.title('Elbow Method for Optimal K (K-Means)')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.xticks(k_range)
plt.grid(True)
plt.show()
```

Figure 3.1: Elbow method for optimal K selection

Next, **Principal Component Analysis (PCA)** is applied as in Figure 3.2 to the selected features to reduce the dimensionality of the data to two principal components. This dimensionality reduction is crucial for visualizing the clusters in a 2D scatter plot as well as undergoing a clustering process to reduce the noise of the data while retaining most of the relevant information from the data.

```
pca = PCA(n_components = 2)
features_pca = pca.fit_transform(combined_features[selected_features])
```

Figure 3.2 : Principal Component Analysis

3.1 K-Means

Following PCA, Based on the figure 3.3, **K-Means clustering** is performed with a predefined number of clusters (set to 3 in this code, likely chosen after observing the elbow method plot, though not programmatically selected). The fit_predict method assigns a cluster label to each data point. These cluster labels are then added as a new column, 'KMeans_Cluster', to the combined_features DataFrame.

```
# Applying KMeans Clustering

kmeans = KMeans(n_clusters = 3, random_state = 42, n_init = 10)
kmeans_labels = kmeans.fit_predict(features_pca)

combined_features['KMeans_Cluster'] = kmeans_labels
```

Figure 3.3 : Performing-Clustering

After we successfully clustered the data, we visualize using a 2d scatter plot using the two principal components via PCA. Each point on the scatter plot is colored according to its assigned K-Means cluster to provide a clear separation between each cluster.

```
features_pca_df = pd.DataFrame(features_pca, columns = ['pc_1', 'pc_2'])
features_pca_df['KMeans_Cluster'] = combined_features['KMeans_Cluster']

plt.figure(figsize = (10, 6))
sns.scatterplot(data = features_pca_df, x = 'pc_1', y = 'pc_2', hue = 'KMeans_Cluster', palette = 'Set2')
plt.show()

# KMeans_Cluster Profiling
original_data['KMeans_Cluster'] = combined_features['KMeans_Cluster']
print(original_data.groupby('KMeans_Cluster')[combined_features.columns].agg(['mean', 'min', 'max']))

profile = original_data.groupby('KMeans_Cluster')['Exam_Score'].agg(['mean', 'min', 'max'])
print(profile)
```

Figure 3.4 : Visualizing and Profiling the Clusters

Finally, cluster profiling is performed as in . The code groups the data with the mean, minimum, and maximum values for each of the selected_features within each cluster. This helps to identify characteristics of each cluster based on the value of the 6 selected features held within clusters.

3.2 Agglomerative

Similar to the K-Means Clustering, Figure 3.5 shows that Agglomerative Clustering is performed. This hierarchical clustering algorithm is performed with a specified number of clusters which is $K = 3$ and a linkage method via ward. The `fit_predict` method assigns cluster labels, which are then added as `Agglomerative_Cluster` to the `combined_features` DataFrame.

```
# Applying Agglomerative Clustering

agglomerative = AgglomerativeClustering(n_clusters = 3, linkage = 'ward')
agglomerative_labels = agglomerative.fit_predict(features_pca)
```

Figure 3.5 : Performing Clustering

Similar with K-Means, the cluster of the Agglomerative Clustering is plotted on 2d scatter plot and **cluster profiling** is performed by adding the Agglomerative cluster labels to the `original_data` and calculating the mean, min, and max for selected_features within each cluster, similar to the K-Means profiling. The function returns the PCA-transformed features.

```
features_pca_df = pd.DataFrame(features_pca, columns = ['pc_1', 'pc_2'])
features_pca_df['Agglomerative_Cluster'] = combined_features['Agglomerative_Cluster']

plt.figure(figsize = (10, 6))
sns.scatterplot(data = features_pca_df, x = 'pc_1', y = 'pc_2', hue = 'Agglomerative_Cluster', palette = 'Set2')
plt.show()

# Agglomerative_Cluster Profiling
original_data['Agglomerative_Cluster'] = combined_features['Agglomerative_Cluster']
print(original_data.groupby('KMeans_Cluster')[combined_features.columns].agg(['mean', 'min', 'max']))

profile = original_data.groupby('Agglomerative_Cluster')['Exam_Score'].agg(['mean', 'min', 'max'])
print(profile)
```

Figure 3.6 : Visualizing and Profiling the Clusters

3.3 Clustering Evaluation

```
def evaluate_cluster(combined_features, features_pca, cluster_name):  
    # Evaluating the cluster using Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Index  
  
    silhouette_scores = silhouette_score(features_pca, combined_features[cluster_name])  
    davies_bouldin_scores = davies_bouldin_score(features_pca, combined_features[cluster_name])  
    calinski_harabasz_scores = calinski_harabasz_score(features_pca, combined_features[cluster_name])  
  
    print("Silhouette Score:", silhouette_scores)  
    print("Davies-Bouldin Index:", davies_bouldin_scores)  
    print("Calinski-Harabasz Index:", calinski_harabasz_scores)
```

Figure 3.7 : Cluster Evaluation Method

Based on the cluster evaluation method above, After finishing the clustering and visualization process we assess the quality of the clusters based on these 3 metrics:

- **Silhouette Score:** This metric measures how similar an object is to its own cluster compared to other clusters. A higher Silhouette Score indicates better-defined clusters.
- **Davies-Bouldin Index:** This index measures the average similarity ratio of each cluster with the cluster that is most similar to it. A lower Davies-Bouldin Index indicates better clustering.
- **Calinski-Harabasz Index:** Also known as the Variance Ratio Criterion, this index is a ratio of the sum of between-clusters dispersion and within-cluster dispersion. A higher Calinski-Harabasz Index generally indicates better-defined clusters.

These evaluation metrics are performed on both the clusters of K-Means and Agglomerative, providing a quantitative basis for comparing the performance of the clustering algorithms.

4.0 RESULT AND DISCUSSION

KMeans Clustering Technique

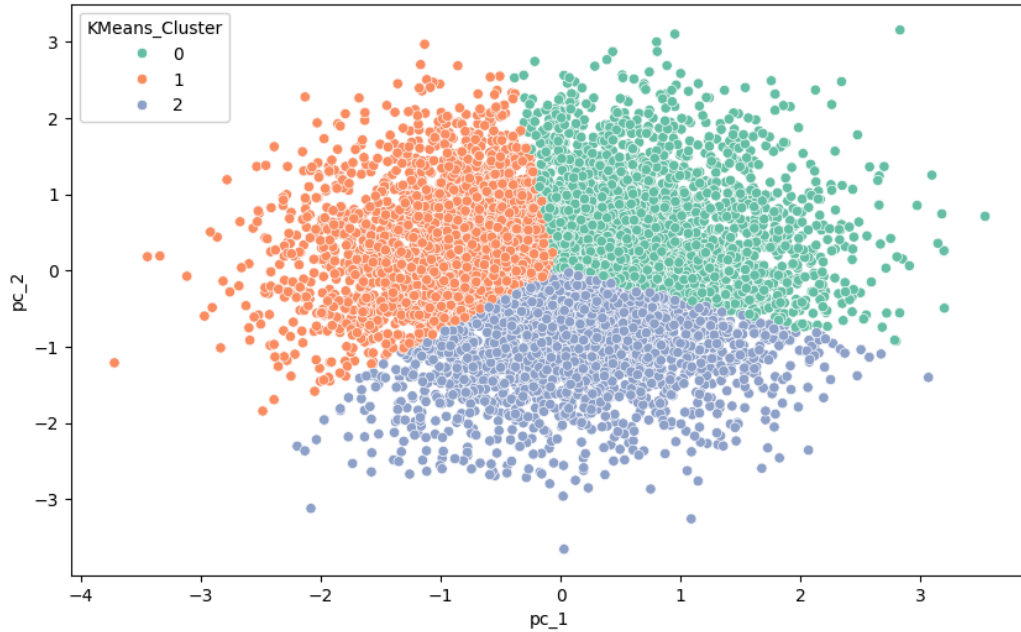


Figure 4.1 KMeans Clustering Graph

Features	Cluster 0	Cluster 1	Cluster 2
Hours Studied (hours per week)	18.63	22.51	18.94
Attendance (%)	87.54	74.19	77.92
Previous Scores (%)	75.95	85.38	64.36
Sleep Hours (hours per day)	6.22	6.97	7.89
Tutoring Sessions (hours per week)	1.54	0.81	1.50
Physical Activity (hours per week)	2.41	3.14	3.31
Exam Score Profile for Each Clusters			
Exam Scores (%)	68.32	66.97	66.09

Table 4.1 Mean Values of Selected Features in Each Cluster (KMeans)

Based on the diagram and table above, a total of three clusters were formed based on the selected features using the KMeans clustering techniques. All of the clusters have the amount of sleep hours that are in the recommended range which is 6 to 8 hours, so this can be excluded as the determining factor of the clusters.

Students from Cluster 0 show the highest average mark among the other 2. The potential features that might affect the result might be the attendance, tutoring sessions, and physical activity that they have per week. Cluster 0 has the best attendance rate and tutoring sessions between the two clusters, allowing them to have a drop in exam score that is not significant. Due to these reasons, they can be labeled as the ‘consistent achievers.’

Cluster 1 has the highest average previous scores but has a lower exam score compared to Cluster 0, showing a significant drop from 85.38% to 66.97%. Factors that contribute to this may be the attendance rate, tutoring sessions, and physical activity per week. Because of the lower attendance rate, the students missed a lot of the class, resulting in falling behind compared to their peers. Additionally, they also lack tutoring sessions to help them catch up with their studies and also can help guide them in their studies. They also might be involved in sports, as the average physical activity done is higher compared to cluster 1. Even though their hours studied per week are the highest, it might not have been enough to cover the topics that were missed in class. Cluster 1 can be labeled as ‘underperforming high potentials’ based on the discussed reasons.

Students from cluster 2 exhibit the lowest exam score of the two at 66.09%. Several features that might contribute to this problem are attendance rate, previous scores, and physical activity. They also have the lowest previous score of the two. Furthermore, their attendance rate is also considered quite low while also having the lowest exam scores among the two clusters. Lastly, the amount of physical activity spent by the students per week is the highest at 3.31 hours per week. This suggests that the students for this cluster might be active in sports, based on the attendance rate causing them to miss a lot of classes and a low exam score suggesting that they have been involved in sports even in previous exams. For cluster 2, they can be labeled as ‘struggling & disengaged.’

Agglomerative Clustering Technique

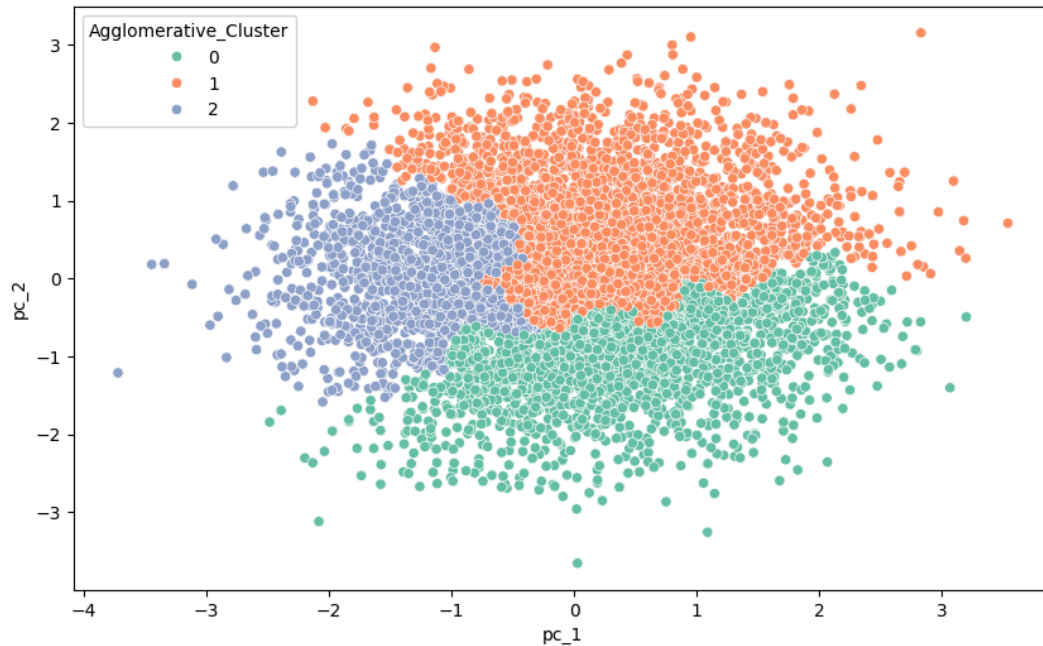


Figure 4.2 Agglomerative Clustering Graph

Features	Cluster 0	Cluster 1	Cluster 2
Hours Studied (hours per week)	18.48	19.48	22.84
Attendance (%)	79.49	84.23	71.58
Previous Scores (%)	62.49	79.18	84.25
Sleep Hours (hours per day)	7.85	6.32	7.34
Tutoring Sessions (hours per week)	1.62	1.29	0.80
Physical Activity (hours per week)	3.25	2.56	3.35
Exam Score Profile for Each Clusters			
Exam Scores (%)	66.22	67.99	66.56

Table 4.2 Mean Values of Selected Features in Each Cluster (Agglomerative)

The Agglomerative Clustering technique was used to create three clusters based on the selected features, as shown in the diagram and table above. Across all clusters, the number of sleep hours is within the healthy or recommended range (7.34-7.85 hours per day). While there are some variances, sleep hours do not appear to be the key differentiating component across these clusters, as all averages are fairly acceptable.

Cluster 0 students have the second-lowest average exam score (66.22%). Their lowest prior scores (62.49%) point to a history of learning challenges. Notably, though, they dedicate the most hours to tutoring on average (1.62 hours per week). Their attendance is fair and their study hours are average, but it seems like they are making an attempt to get guidance. Their high hours spent on physical activity per week may also be a sign of conflicting schedules. Despite continuous difficulties, this group seems to be actively addressing their weaknesses. Thus, this cluster can be labeled as ‘struggling but proactive’.

Cluster 1 has the highest average exam result (67.99%). This group stands out for its excellent attendance rate (84.23%) and strong previous scores (79.18%). These criteria, taken together, suggest a profile of sustained academic involvement and basic strength. Their reasonable study hours and tutoring sessions, paired with the lowest physical activity among the clusters, are most likely contributing to their overall greater academic performance. They can be labeled as ‘consistent high achievers’.

Students in Cluster 2 get the lowest exam score (66.09%). Despite having the most study hours per week (22.84 hours) and the highest previous results (84.25%), they have a significant reduction in current exam performance. This shows that their significant studies may be ineffective. The main contributing variables are their low attendance rate (71.58%) and limited participation in tutoring sessions (0.80 hours per week). Their maximum physical activity is likely due to extensive time spent on non-academic pursuits, which is worsened by their low involvement in educational channels, resulting in a gap between their high potential/effort and their actual academic performance. This cluster can be labeled as ‘high-potential underperformers’.

Clustering Algorithm Evaluation

Evaluation Metrics	K-Means Clustering	Agglomerative Clustering
Silhouette Score	0.3388	0.2927
Davies-Bouldin Index	0.9291	1.0293
Calinski-Harabasz Index	3847.3226	3145.8825

Table 4.3 Evaluation of Clustering Algorithms

K-Means Clustering had a Silhouette Score of 0.3388, which was more than Agglomerative Clustering's 0.2927. This shows that, K-Means clusters are slightly more distinct and well-separated from one another, with data points being more cohesive inside their respective cluster. While these scores show some discernible structure, they are in the "fair" category, indicating that clusters exist but are not completely isolated. K-Means Clustering produced a Davies-Bouldin Index of 0.9291, which is less than Agglomerative Clustering's 1.0293. Having a lower Davies-Bouldin Index is beneficial. This finding validates K-Means' demonstrate that it created more compact and well-separated clusters. K-Means Clustering produced a Calinski-Harabasz Index of 3847.3226, which was much greater than Agglomerative Clustering's 3145.8825. A higher Calinski-Harabasz Index suggests improved grouping.

Derived Business Insights

1. K-Means Cluster 1/Agglomerative Cluster 2 "Underperforming High Potentials" should be identified and supported. A subset of students (K-Means Cluster 1, Agglomerative Cluster 2) have a history of good test scores and/or substantial study time, yet they do noticeably worse on exams now. This implies an imbalance between their effort and potential and their results. Despite their fundamental strengths and commitment to learning, interventions should concentrate on resolving issues like low attendance and a lack of tutoring sessions, which seem to be limiting their achievement. Mandatory attendance, individualized academic counseling, and promoting involvement in extracurricular learning materials are a few examples of targeted support.

2. For "Struggling & Disengaged" and "Struggling but Proactive" students (K-Means Cluster 2 / Agglomerative Cluster 0), create targeted programs. In addition to having poor attendance and high levels of physical activity, one student group (K-Means Cluster 2) routinely exhibits the lowest exam and prior scores, suggesting disengagement or conflicting commitments. Agglomerative Cluster 0 is another group that actively seeks tuition despite having low exam and prior scores, indicating a proactive but continuing to struggle. In order to motivate them and enhance their performance, developing customized assistance programs, which may include required academic support sessions, closer supervision, and even flexible learning alternatives or sports-friendly academic timetables.
3. Use "Consistent Achievers" (K-Means Cluster 0 / Agglomerative Cluster 1) for Peer Mentoring and Best Practices: Strong attendance, high prior scores, and steady present exam performance are characteristics of the clusters designated as "consistent achievers" (K-Means Cluster 0, Agglomerative Cluster 1). By sharing their effective study techniques and engagement tactics with other students who are having difficulty, these individuals could be extremely beneficial members of peer mentorship programs.

6.0 CONCLUSION

According to the findings, K-Means and Agglomerative Clustering were effective tools for analysing student performance, showing unique student groupings and offering useful data for focused solutions.

Key recommendations include:

- **Targeted Support:** Give "Underachievers" (bad test results, low attendance, low motivation) personalised academic counselling and extracurricular resources.
- **Customized Assistance:** Provide "Struggling & Disengaged" and "Struggling but Proactive" students (poor attendance, low exam/prior scores, and some requesting tuition) with the necessary academic help and closer supervision.
- **Peer Mentoring:** To exchange efficient study methods, use "Consistent Achievers" (high attendance, high past scores, and consistent performance) for peer mentorship programs.

7.0 APPENDIX

Unsupervised Learning - Clustering Code Implementation

REFLECTION

IMAN ABADI

PART A: PROJECT REFLECTION

The first project was on supervised learning (classification). The first step was data understanding, where we explored the dataset that we had chosen to gain familiarity with the data's characteristics, including its quality, structure, and potential issues. We also came up with 6 business insights based on the chosen dataset. We had trouble in determining business insights as to how it is related to the project. An example question raised was "Do we have to use all of the features or only features related to the target label of what we wanted to predict?" Next, we executed the preprocessing and sampling step, where we cleaned the data to remove duplicate values, manage null values, and correct data discrepancy. We faced the most challenge in this step, as there was a lot of trial and error. We explored a lot of ways to process our data so that it can be fitted into a model properly to yield the best result. We also transformed the data into a suitable format using target encoding and then scaled the data to normalize it. Before the data was fit into the model, we made sure to use the stratified sampling technique to ensure that the training set of the data is proportionate to the testing set. After the model was trained, each of us provided the evaluation score of our classification algorithm for comparison. We compared the scores of each evaluation metric, which were accuracy, F1 score, precision, and recall for each insight, and then discussed why the values differ from each other. The evaluation on one insight for all algorithms was good, but the other two proved to be unsatisfactory. This was due to the imbalanced data of our selected feature to be labeled that was in the first insight, causing our model to be biased. From this project, I have learned various ways to preprocess data, using extensive Python libraries that have multiple different approaches. I also learned about my chosen algorithm and also learned ways to evaluate a model using different evaluation metrics. Additionally, I also learned how and why each algorithm performs differently than other algorithms, as some algorithms rely heavily on how the data is preprocessed, such as noise in a dataset. I also learned the significance of data understanding, as it saves you from the need of handling issues in the preprocessing and model fitting step.

The second project was on unsupervised learning (clustering). We used the same dataset, but the preprocessing steps are slightly different. This step also proved to be a challenge for us, as there was a lot of trial and error. At first, we used the same preprocessing steps that were done in the previous project, but the results were unsatisfactory. To overcome this issue, we searched for ways to produce better results. We approached this issue by applying what was mentioned in the first project but with additional steps involved. First, we removed outliers from the dataset, as clustering algorithms are sensitive to outliers. Then, we use the feature. selection to determine the best features that yield the best clustering results. This yielded the best result from our trials and errors. Then, the algorithm was fitted, and the clusters are then shown in a scatter plot graph. At first the clusters are labeled, but the actual 'cluster' was not clearly seen in the graph. Then we realized that we had to do Principal Component Analysis (PCA) to reduce the dimensionality of the data before we could visualize the clusters in a 2D scatter plot. After the clusters were formed, evaluation metrics for clustering algorithms were used to determine the performance of the algorithms. Finally, it was labeled according to the profile of the clusters that was formed. From this project, I learned that not all machine learning tasks use the same preprocessing approach. I also learned how feature selection works to improve the performance of a clustering algorithm.

PART B: COURSE REFLECTION

This Data Mining course has been an interesting experience for me, as it is related to my subject of interest in computer science, which is data science. It provides a solid foundation in the principles and practical applications of data mining and machine learning. The hands-on projects, which required dealing with the complexity of real-world datasets, were especially helpful in strengthening my understanding and building necessary skills. I believe I have effectively accomplished the learning outcomes by getting a thorough understanding of various data mining tasks (supervised and unsupervised learning) and knowing each critical step in carrying out a data mining task. My skills for data analysis were greatly increased from this course. I became knowledgeable in using Python libraries for preprocessing, data manipulation, and model development. Proficiency in dimensionality reduction and feature selection, the ability to apply and assess both classification and clustering methods, and advanced data cleaning and transformation techniques are among the key skills gained. My critical thinking and troubleshooting skills were also refined by the iterative problem-solving that the projects required. The abilities learned in this course are directly transferable and will be extremely useful in my future academic and career pursuits. Academically, I plan to use my organized approach to data analysis and better Python expertise for advanced studies and research. Professionally, I am now prepared to contribute to data-driven decision-making by resolving real-world business challenges, streamlining processes, and successfully presenting complicated insights. This course has provided a solid basis for a profession focused on harnessing data to create meaningful solutions.

MUHAMMAD ADAM BIN RAZALI

PART A: PROJECT REFLECTION

For the Project Phase 1, There are several key things that I learned from it which are the deeper understanding of how important data preprocessing is to develop a decent machine learning model. In the preprocessing itself, major challenges is to learn how to Handle missing values, how to convert categorical and continuous values into meaningful data to feed inside the machine learning model for training and testing phase, especially on deep learning which consist of multiple linear layers that passed data through it by forward passing and learning it through backward propagation and gradient descent catch patterns of the data seamlessly. I also learn that not every model of machine learning suits for every data, some are sensitive to distance measurement of the data, some can handle continuous data really well. There were also a hard time for me to figure out the form of the data that can be suited inside the model such that should it be Dataframe, Numpy or just a normal array. Despite all that, the classification task using deep learning further solidified my grasp of neural network principles, including model architecture design, training optimization, and evaluating performance metrics, showcasing the power of deep learning in tackling complex classification problems after thorough data preparation.

For the project phase 2, I learned to improvise my data preprocessing step to adapt into the clustering process via K-Means Clustering, and Agglomerative Clustering. I learned to handle missing values even better and to substitute outliers instead of just removing them, but figuring out about the feature selection and dimensionality reduction was the hard part. Eventually we manage to tackle it via Principal Component Analysis and Variance Threshold that massively reduce noise and inaccuracy of the clusters generated by the clustering algorithm. Essentially, I learn to improve the data qualities before doing analytical summaries to make sure it gave the best representation of the data. Besides that, my work with clustering algorithms, specifically K-Means and Agglomerative, allowed me to explore unsupervised learning techniques for uncovering hidden patterns and natural groupings within the data. This practical application reinforced the importance of selecting appropriate algorithms and evaluating their effectiveness in real-world scenarios.

Most importantly, in every group working, I learn to communicate my ways to my fellow group mates to deliver great understandings of my ideas to them, which in general boosts my soft skill as well as my communication skill. Making the business insight were also struggling since lots of factors need to be taken into account.

PART B: COURSE REFLECTION

Reflecting on the Data Mining Course, I've come to appreciate its rich history and evolution. From its early roots in the 1960s, where manual coding and statistical methods were the norm, to the sophisticated techniques and tools we use today, data mining has transformed significantly. The emergence of Knowledge Discovery in Databases (KDD) in the 1990s marked a pivotal shift, emphasizing the extraction of valuable, non-trivial patterns from data. The development of specialized conferences and tools further solidified data mining as a critical field within computer science, driven by the increasing availability of large datasets and the need to extract actionable insights.

Understanding the distinction between supervised and unsupervised learning has been crucial. Supervised learning, with its focus on predicting outcomes based on labeled data, contrasts sharply with unsupervised learning, which seeks to uncover hidden patterns in unlabeled data. This difference highlights the versatility of data mining techniques, allowing us to approach diverse problems from classification and regression to clustering and association rule mining. The data preprocessing stage, involving cleaning, transformation, and feature selection, is essential for ensuring the quality of the data used in both types of learning.

Finally, exploring clustering and association rules has provided practical insights into how data mining can be applied. Clustering algorithms, like K-Means and Agglomerative, enable us to group similar data points, revealing underlying structures. Association rule mining helps identify relationships between variables, such as market basket analysis, which has broad applications in business and beyond. These techniques, combined with a solid understanding of data preprocessing, form a powerful toolkit for extracting meaningful knowledge from complex datasets.

MUHAMMAD AFIQ DANISH BIN MOHD HAZNI

Reflection:

Personally, I thought the Data Mining course was kind of boring and easy to learn. But the truth is it is challenging and interesting at the same time. I really like to learn how to handle data, manipulate and transform data into a quality one, but I did not know how to do it. In the previous semester, I learned about Database but it is the most basic thing about what I need to know for Data. So my knowledge in handling data was poor and sometimes confusing. In the Data Mining course, I got to learn about data preparation and pre-processing, and how to turn raw data into something meaningful. It is my favourite part of the course. I learned various techniques such as handling missing values, removing duplicates and binning continuous features. I acquired a new skill during the process. So, I will know what to do when I get a raw dataset to be studied and analyzed in the future. Additionally, I also learned about two types of machine learning, supervised and unsupervised. In Data Mining, we use machine learning to build a model for us to predict the data. There are two types which I mentioned before, and below those types there are tons of algorithms that I learned. My favourite is supervised learning because it is not complex like unsupervised. Supervised learning models learn from the past events which are input and class labels. While unsupervised, the model only learns based on input and there are no labels, which might be a little bit tricky. In supervised learning, I got to do classification on my chosen dataset in project 1. I use the SVM algorithm to do the classification technique, and gradually understand how it works upon project completion. It is interesting to know that I successfully built a machine learning model for the first time in my life. Although there are tons of algorithms in this topic, I learned some of them such as K-Nearest Neighbors (KNN), Deep Learning, Artificial Neural Network (ANN) in this class. I will try to do these algorithms by myself in the future so that I can understand more. Not to mention, I also got to use unsupervised learning which is clustering techniques in my project 2 with the same dataset. My team uses K-means and Agglomerative clustering to cluster the data. Even though my understanding for unsupervised learning is quite poor, it does not stop me from learning more by myself about this topic. I experienced how to implement both of these algorithms using Python, as it is easier for me rather than using tools. The libraries and built in functions that Python provides help me to understand

how the algorithms are implemented and also enhance my Python skills. Furthermore, I also learned about Association Rules Mining which is another type of unsupervised learning. This technique is easy to understand, I learned two algorithms which are Apriori and FP-Growth. This technique is usually being implemented for transactional data, to identify relationships between sets of items. For instance, what are the items that are frequently bought together. It is easier to understand compared to classification and clustering. But I do not get the chance to implement the algorithm technically like the others, only make a poster about it. So I think it might not help me to understand the algorithm best. Overall, this course has taught me a lot of skills that I need and it is fun. The lecturer is sporting and easy to approach, which also helps a lot when I have a question to ask. I am glad to have the opportunity to learn about this course. By using all of the skills that I have acquired, I will apply it to my work and help my organization to profit more. For example, if I were hired into a grocery company, I would use association rules mining to determine which of their products that are frequently bought together, and help the store manager to organize the product in such a manner that customers will find those products easily. Thus, the profit of the company will increase and my experience will also increase. Not to mention, it is the same for other techniques depending on what type of data that I will be handling. I am grateful to acquire all of this knowledge.